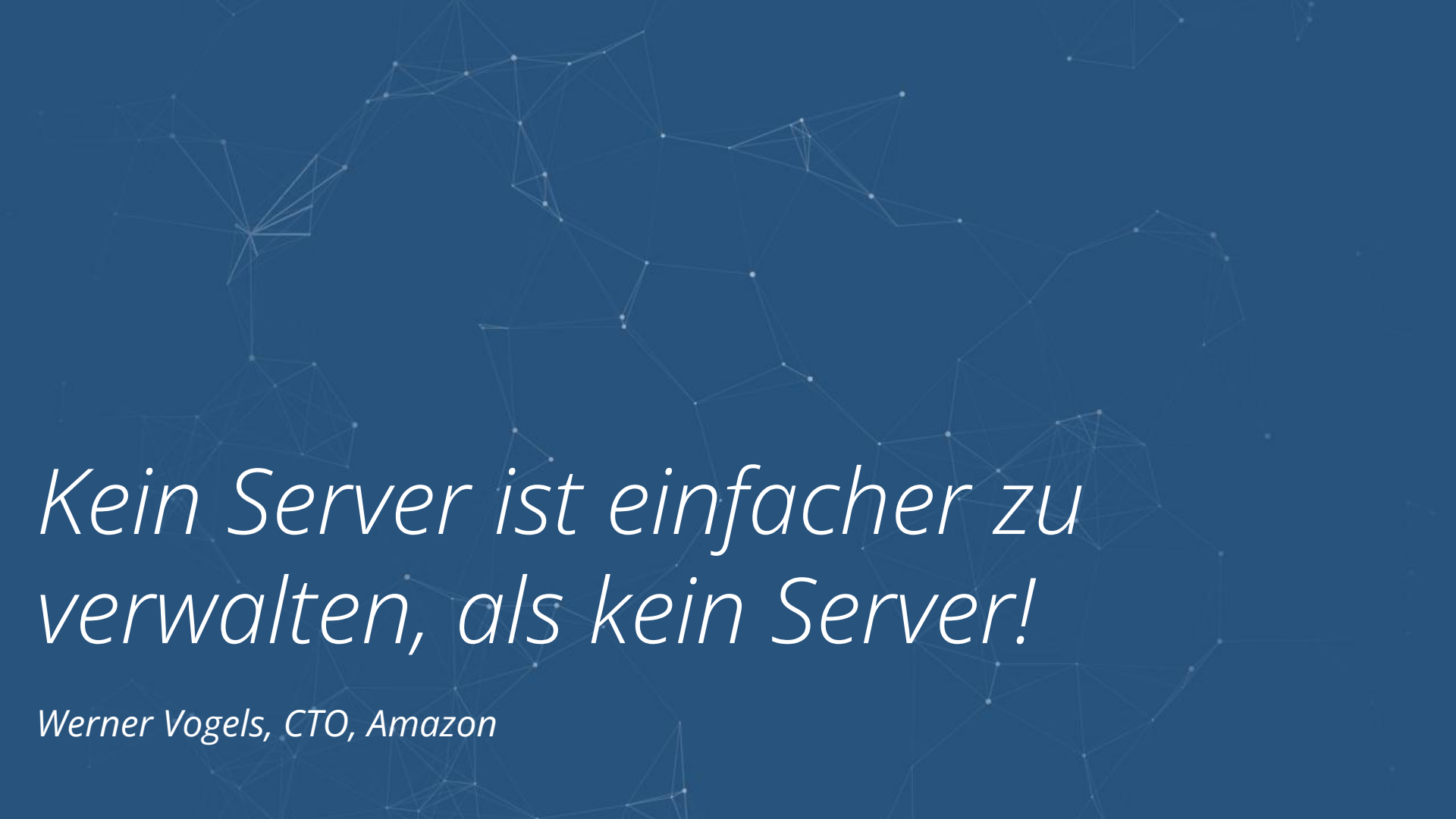


Cloud Computing Serverless

The background is a solid dark blue color. Overlaid on this is a complex, abstract network of thin, light blue lines connecting small, light blue dots. These dots and lines are scattered across the entire frame, creating a sense of a global or digital network. The lines vary in length and orientation, forming a web-like structure.

*Kein Server ist einfacher zu
verwalten, als kein Server!*

Werner Vogels, CTO, Amazon



The background is a solid dark blue color. Overlaid on this is a complex, abstract network of thin, light blue lines connecting small white dots. These dots and lines are scattered across the entire frame, creating a sense of a global or digital network. The lines vary in length and orientation, forming a web-like structure.

Ist Cloud Native nicht
immer Serverless?

CLOUD NATIVE TRAIL MAP

The Cloud Native Landscape *Landio* has a large number of options. This Cloud Native Trail Map is a recommended process for leveraging open source, cloud native technologies. At each step, you can choose a vendor-supported offering or do it yourself, and everything after step #3 is optional based on your circumstances.

HELP ALONG THE WAY

A. Training and Certification

Consider training offerings from CNCF and then take the exam to become a Certified Kubernetes Administrator or a Certified Kubernetes Application Developer

cncf.io/training

B. Consulting Help

If you want assistance with Kubernetes and the surrounding ecosystem, consider leveraging a Kubernetes Certified Service Provider

cncf.io/csp

C. Join CNCF's End User Community

For companies that don't offer cloud native services externally

cncf.io/enduser

WHAT IS CLOUD NATIVE?

Cloud native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds. Containers, service meshes, microservices, immutable infrastructure, and declarative APIs exemplify this approach.

These techniques enable loosely coupled systems that are resilient, manageable, and observable. Combined with robust automation, they allow engineers to make high-impact changes frequently and predictably with minimal toil.

The Cloud Native Computing Foundation seeks to drive adoption of this paradigm by fostering and sustaining an ecosystem of open source, vendor neutral projects. We democratize state-of-the-art patterns to make these innovations accessible for everyone.

l.cncf.io

v20200501



1. CONTAINERIZATION

- Commonly done with Docker containers
- Any size application and dependencies (even PDP-11 code running on an emulator) can be containerized
- Over time, you should aspire towards splitting suitable applications and writing future functionality as microservices

3. ORCHESTRATION & APPLICATION DEFINITION

- Kubernetes is the market-leading orchestration solution
- You should select a Certified Kubernetes Distribution, Hosted Platform, or installer *cncf.io/ckd*
- Helm Charts help you define, install, and upgrade even the most complex Kubernetes application



5. SERVICE PROXY, DISCOVERY, & MESH

- CoreDNS is a fast and flexible tool that is useful for service discovery
- Envoy and Linkerd each enable service mesh architectures
- They offer health checking, routing, and load balancing



7. DISTRIBUTED DATABASE & STORAGE

When you need more resiliency and scalability than you can get from a single database, Vitess is a good option for running MySQL at scale through sharding. Rocks is a storage orchestrator that integrates a diverse set of storage solutions into Kubernetes. Serving as the "brain" of Kubernetes, etcd provides a reliable way to store data across a cluster of machines. TKV is a high performance distributed transactional key-value store written in Rust.



9. CONTAINER REGISTRY & RUNTIME

Harbor is a registry that stores, signs, and scans content. You can use alternative container runtimes. The most common, both of which are OCI-compliant, are containerd and CRI-O.



2. CI/CD

- Setup Continuous Integration/Continuous Delivery (CI/CD) so that changes to your source code automatically result in a new container being built, tested, and deployed to staging and eventually, perhaps, to production
- Setup automated rollouts, roll backs and testing
- Argo is a set of Kubernetes-native tools for deploying and running jobs, applications, workflows, and events using GitOps paradigms such as continuous and progressive delivery and MLops



4. OBSERVABILITY & ANALYSIS

- Pick solutions for monitoring, logging and tracing
- Consider CNCF projects Prometheus for monitoring, Fluentd for logging and Jaeger for tracing
- For tracing, look for an OpenTracing-compatible implementation like Jaeger



6. NETWORKING, POLICY, & SECURITY

To enable more flexible networking, use a CNI-compliant network project like Calico, Flannel, or Weave Net. Open Policy Agent (OPA) is a general-purpose policy engine with uses ranging from authorization and admission control to data filtering. Falco is an anomaly detection engine for cloud native.



8. STREAMING & MESSAGING

When you need higher performance than JSON-RPC, consider using gRPC or NATS. gRPC is a universal RPC framework. NATS is a multi-modal messaging system that includes request/reply, pub/sub and load balanced queues. CloudEvents is a specification for describing event data in common ways.



10. SOFTWARE DISTRIBUTION

If you need to do secure software distribution, evaluate Notary, an implementation of The Update Framework.



Filter: GROUP Projects and products Members Certified partners and providers Services Watch CHN VIM KATO Git Cad TIOBA +

Application Software & Image Host

Database

Container Integration & Delivery

Security & Management

Scheduling & Orchestration

Service Proxy

API Gateway

Service Mesh

Remote Function Call

Coordination & Service Discovery

Cloud Native Storage

Cloud Native Network

Container Runtime

Security & Compliance

Automation & Configuration

Container Registry

Key Management

Observability

Container Optimization

Chaos Engineering

Feature Flagging

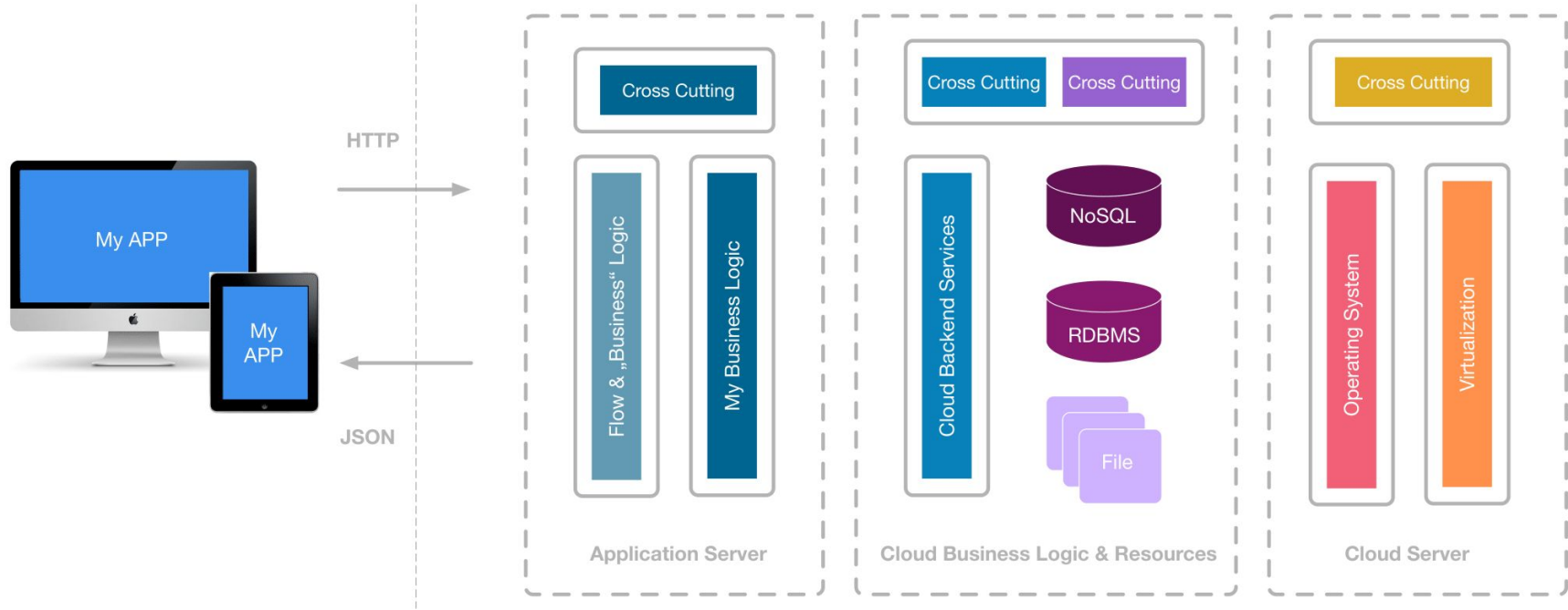


Cloud Native ist

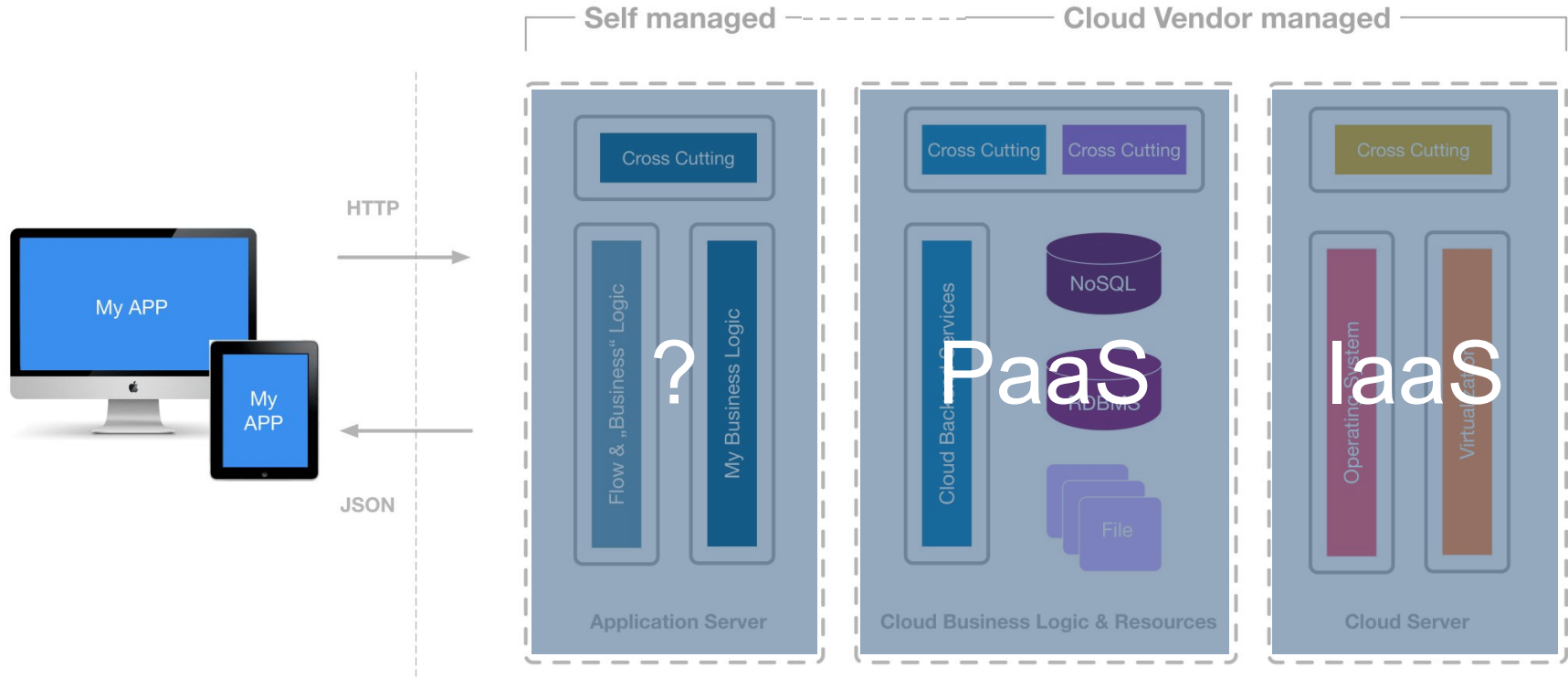
komplex!

Kubernetes, Docker, Microservices, YAML, ...

Traditionelle Cloud-basierte Anwendungsarchitektur



Traditionelle Cloud-basierte Anwendungsarchitektur



Serverless computing refers to a new model of cloud native computing,

Serverless computing refers to a new model of cloud native computing, enabled by architectures that do not require server management to build and run applications.

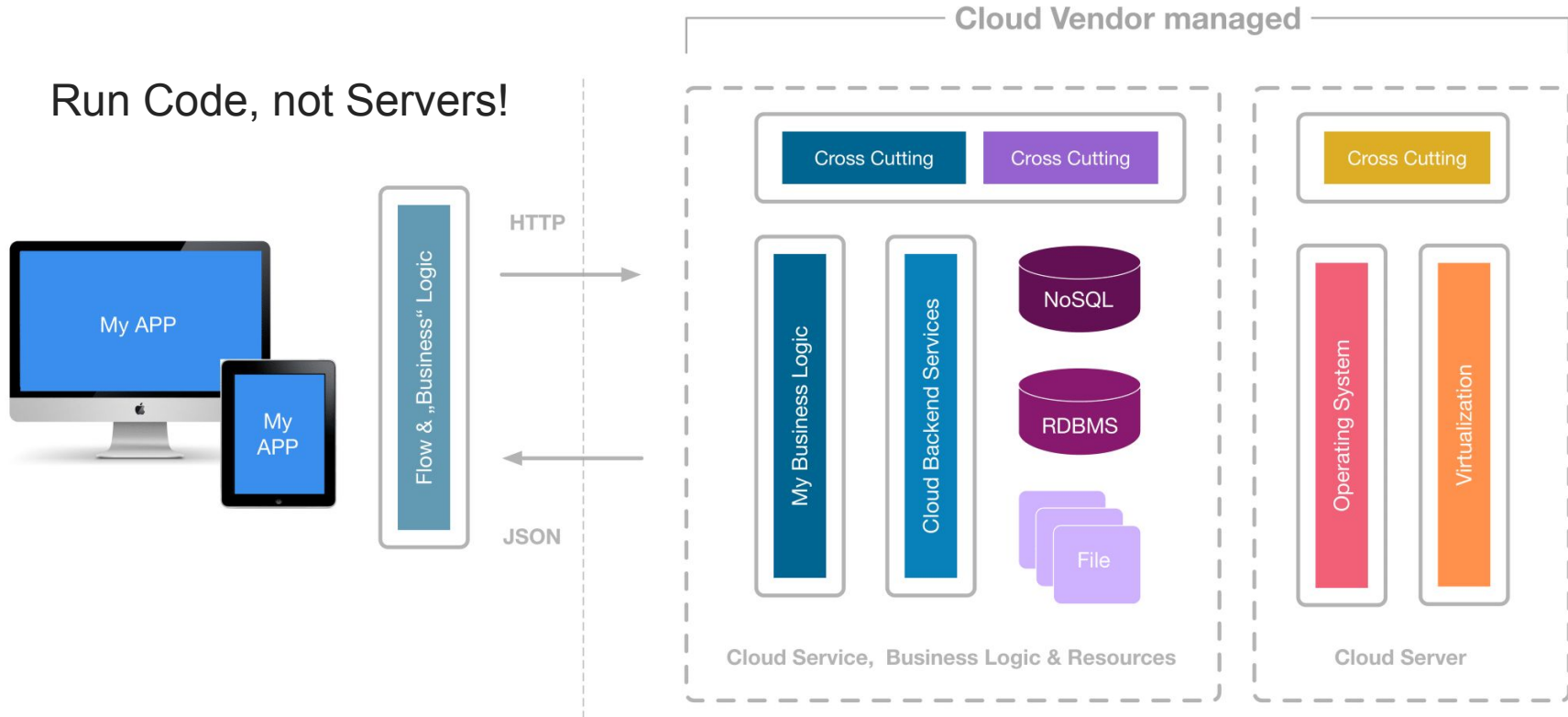
Serverless computing refers to a new model of cloud native computing, enabled by architectures that do not require server management to build and run applications. It leverages a finer-grained deployment model

Serverless computing refers to a new model of cloud native computing, enabled by architectures that do not require server management to build and run applications. It leverages a finer-grained deployment model where applications, bundled as one or more functions, are uploaded to a platform

Serverless computing refers to a new model of cloud native computing, enabled by architectures that do not require server management to build and run applications. It leverages a finer-grained deployment model where applications, bundled as one or more functions, are uploaded to a platform and then executed, scaled, and billed in response to the exact demand needed at the moment.

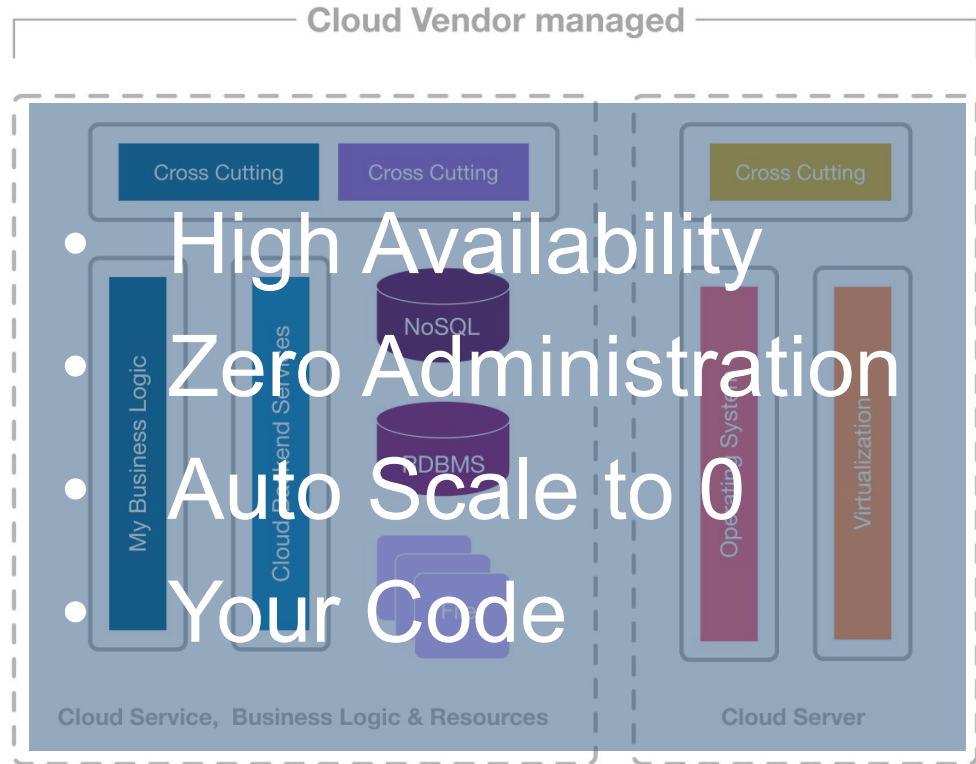
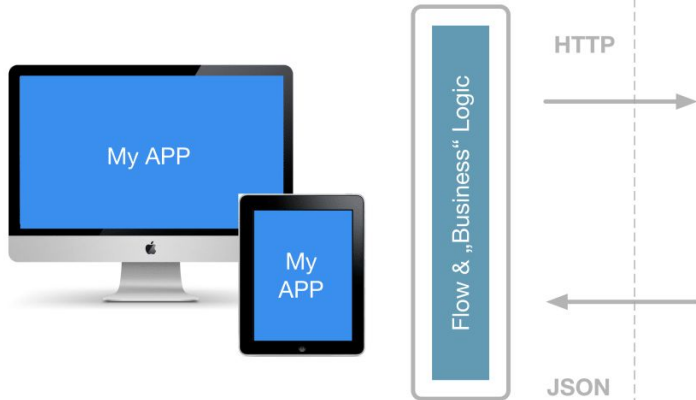
Serverless – Anwendungsarchitektur

Run Code, not Servers!

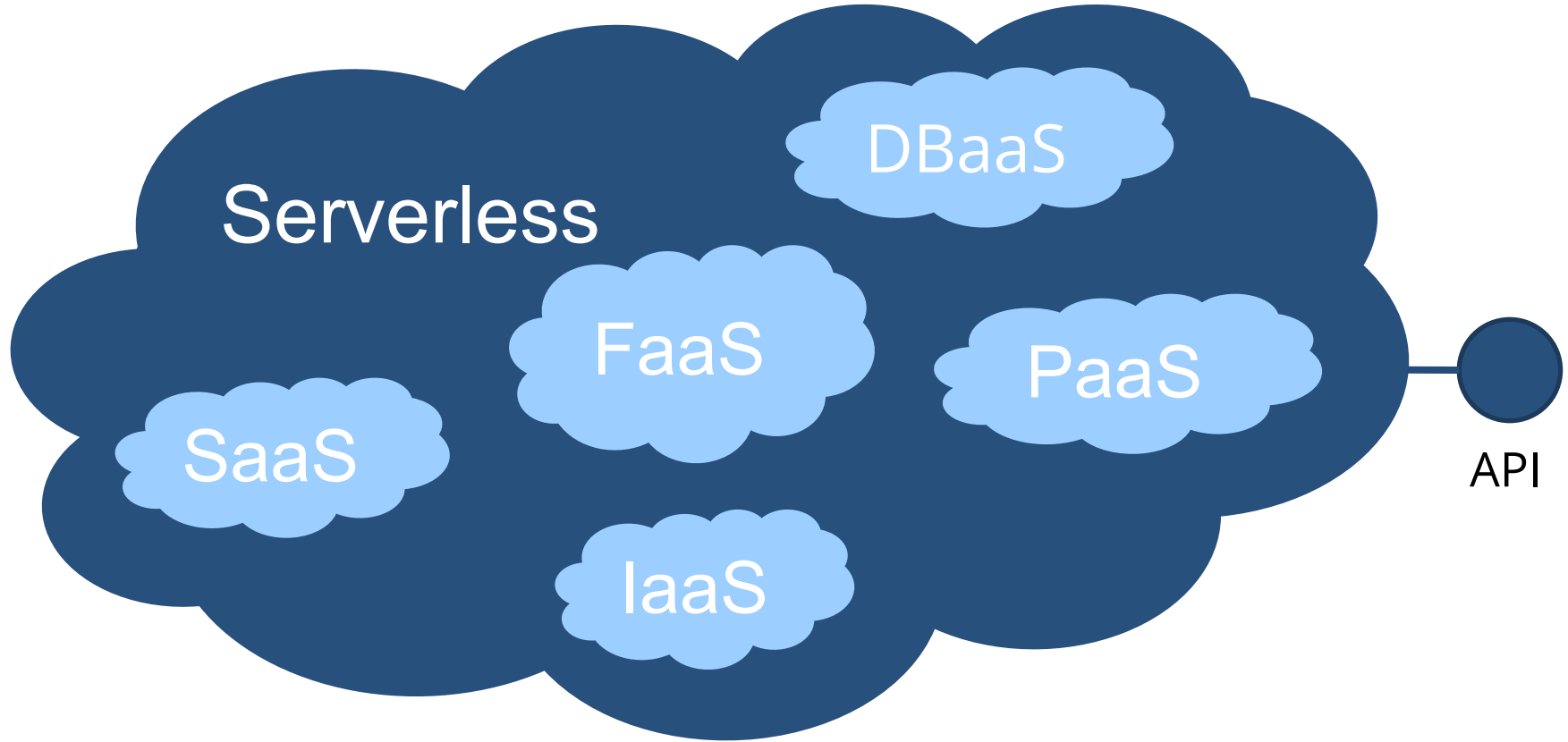


Serverless – Anwendungsarchitektur

Run Code, not Servers!



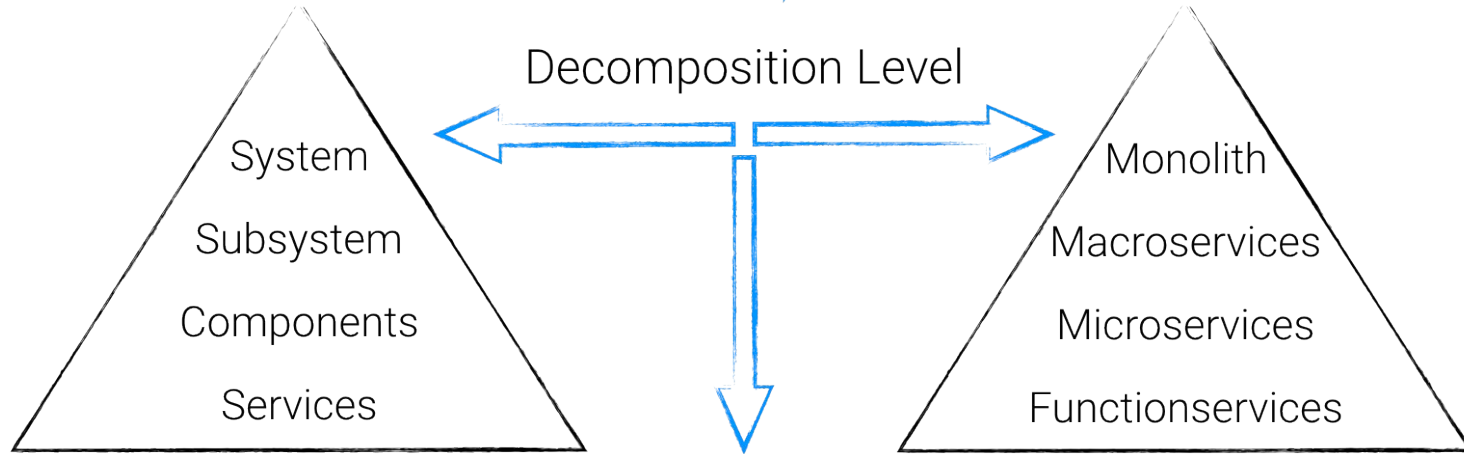
Out-of-the Box Self-scaling Fully Managed Backend



Dev Components



Ops Components



Decomposition Trade-Offs

- + More flexible to scale
- + Runtime isolation (crash, slow-down, ...)
- + Independent releases, deployments, teams
- + Higher resources utilisation

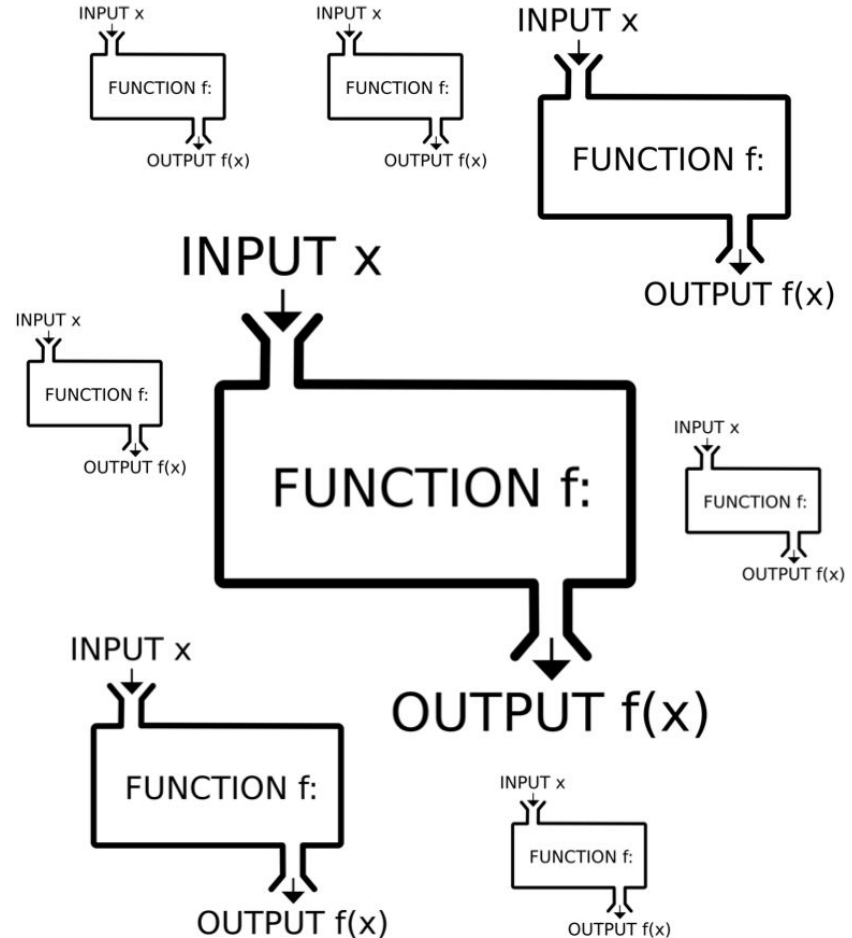
- Distribution debt: Latency, Consistency
- Increased infrastructure complexity
- Increased troubleshooting complexity
- Increased integration complexity

Serverless-Programmiermodell:

Functions

(as a Service)

= FaaS

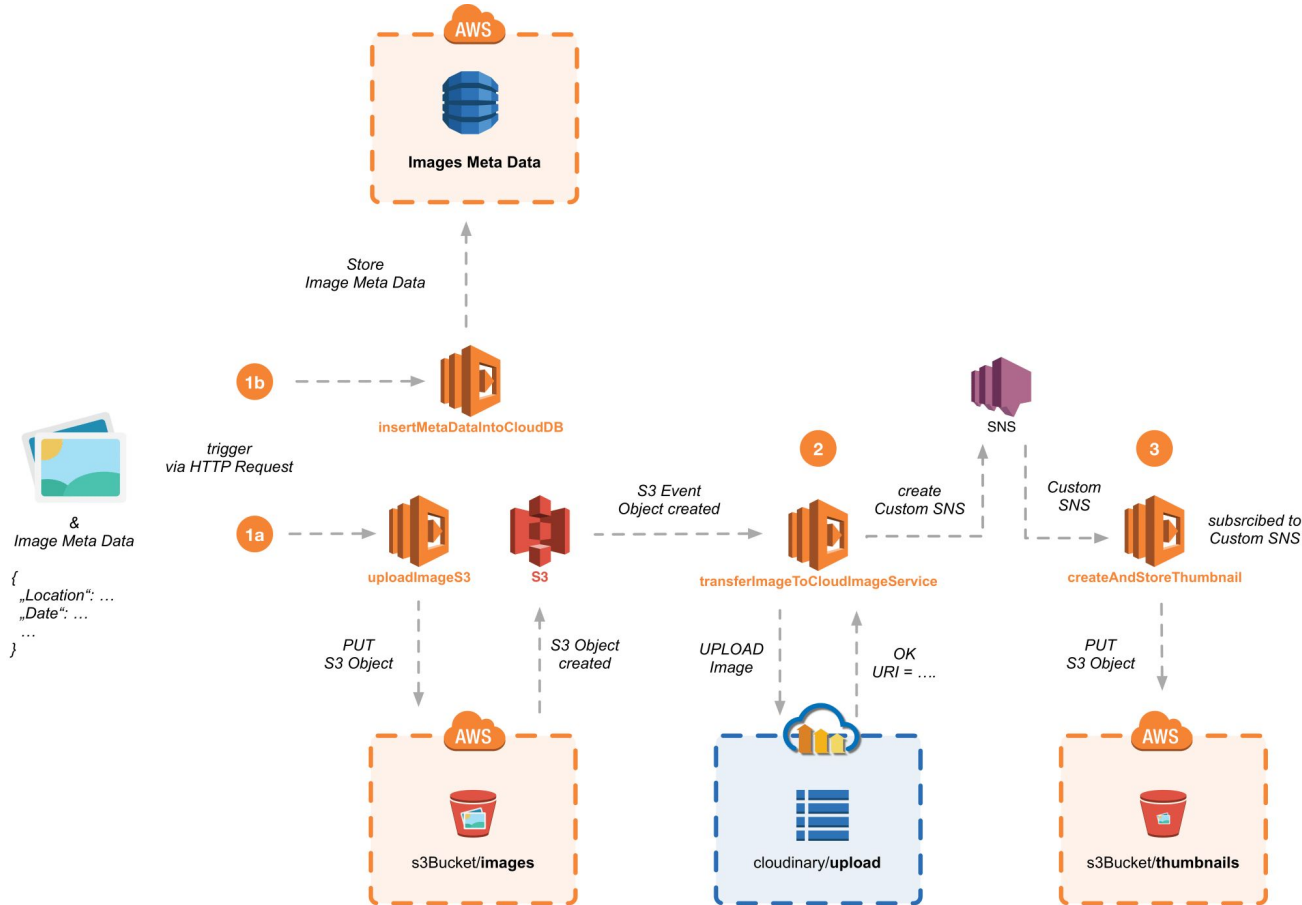


FaaS ist perfekt für Event-getriebene Architekturen -
es ermöglicht

lose gekoppelte, reaktive
Komponenten oder Services.

Achtung: Kostenmultiplikator (ca. $\sim$ Faktor 4) bei hoher Last

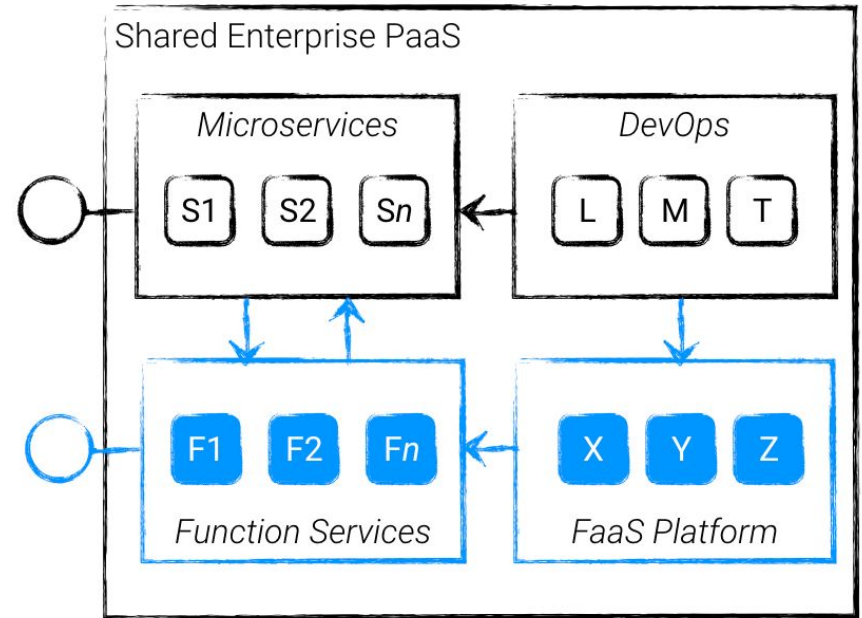
Thumbnails erstellen à la AWS Lambda



Use Case 1

Hybrid Architectures

- Kombination von Microservice Architektur mit EDA
- Nutzung von Function Services für Event-getriebene Use Cases
- Reduzierter Ressourcen-Verbrauch per Scale-to-Zero
- Integration in bestehende Enterprise PaaS Umgebung

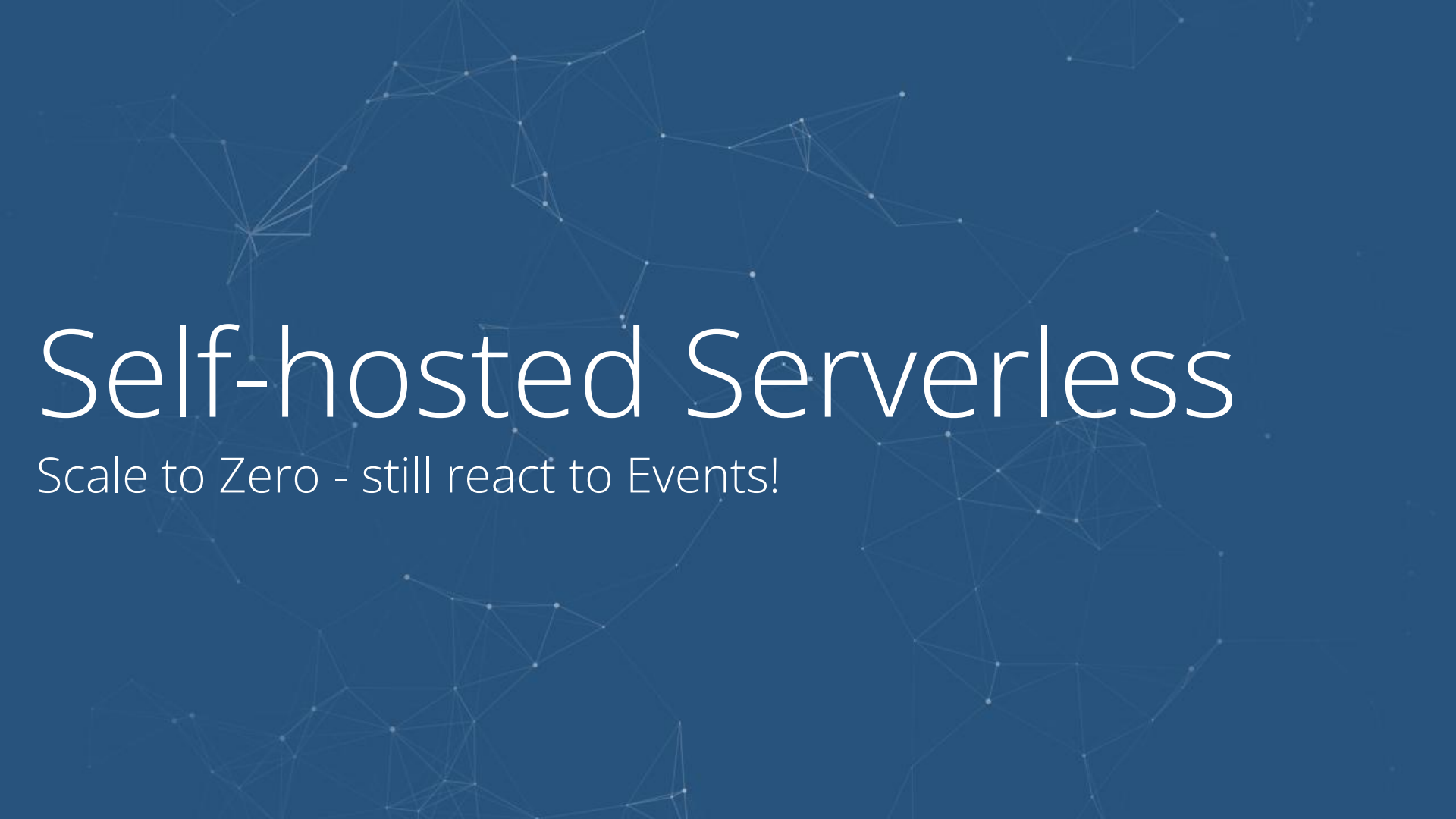


Use Case 2

Edge und Fog Computing

- Anbindung der LoRaWan Raum-Sensoren mittels Serverless Backend
- Couch Projekt: Nutzung von FaaS auf Low Power Devices
- Unterstützung von leichtgewichtigen Cluster Scheduler wie Docker Swarm





Self-hosted Serverless

Scale to Zero - still react to Events!

Filters

GROUP:

Projects and products

Members

Certified partners and providers

Serverless

Wasm

CNAI

VIEW MODE:

Grid

Card

ZOOM:

-

+

Serverless

Installable Platform 🔍



Tools 🔍



Installable Platforms 🔍



Hosted Platform 🔍



Embedded Functions 🔍



Framework 🔍





OPENFAAS



fission



nuclio



Die Kandidaten

- OpenWhisk
<https://openwhisk.apache.org/>
- OpenFaas
<https://www.openfaas.com>
- Fission
<https://fission.io>
- Nuclio
<https://nuclio.io>
- Knative
<https://knative.dev/>
- Keda
<https://keda.sh/>

siehe auch <https://bit.ly/2Mh1kxj>

Knative



Knative is a platform-agnostic solution for running serverless deployments.

- FaaS: Knative Functions
 - Eine Vielzahl von Programmiersprachen wird hier durch Language Packs unterstützt:
<https://github.com/knative/func/blob/main/docs/language-packs/language-pack-contract.md#built-in-language-packs>
- Auto Scaling: Knative Serving
- EDA: Knative Eventing

KEDA



- Kubernetes Autoscaling/Event Platform
- Kann für EDAs verwendet werden
- Scaling/Erzeugen von Containern ist mit einer Vielzahl von Triggern möglich: Prometheus Metriken, Queues (SQS, Kafka, ...), Databases (Postgres, MySQL, ...)

